开发过程说明

1. 从业务到技术的拆解

业务目标：从学生作答估计「每名学生 × 每个知识点」的掌握概率，并用中文界面讲清楚。拆成三层：

- **数据层**：Q 矩阵（题 × 知识点）与 X 作答矩阵（学生 × 题）以长表落 SQLite，共 8 张表。合成数据固定种子可复现：24 题 = 15 单技能（每个知识点 3 道，含仅测 KP5 的三道）+ 9 综合；X 为 12 × 24 = 288 条完整作答。 seed 建库后另写一份可核验报告 <库名>.report.txt，记录收敛标志、迭代轮数与每轮对数似然。seed 按「data_gen 造数 → dina 校准 → db 落库」的顺序一次性编排，默认拒绝覆盖、--force 显式重建。
- **算法层**：DINA 把作答映射到 32 种掌握画像，理想作答是 And 门（须掌握题目所需全部知识点）；在对数域 做画像后验与 EM 题参校准。掌握概率是画像后验的逐技能边际，不是答对率，也不由合成真值 profile_id 生成。
- **展示层**：只读 HTTP 服务提供 6 个 GET 接口（meta/students/students/{id}/graph/items/diagnosis），前端仪表盘主要用学生列表、单生诊断、图谱与全体诊断矩阵，渲染雷达图、概率对照表、前置图谱、12×5 矩阵； items（Q 矩阵与题参）保留为透明校验接口，不塞进主界面。接口为只读 JSON，错误统一为 {error:{code,message}}，静态资源按白名单路由并做路径穿越防护。前端做到可访问：label 关联、可见键盘焦点、 中文加载/错误/重试、快速切换用 AbortController 防旧结果覆盖、390px 无横向溢出。

关键取舍：知识图谱只解释学习顺序、不改变推断；真值仅用于造数与验证、不经 API 暴露；应用零第三方运行依赖 （Python 标准库 + SQLite + 原生 HTML/CSS/JS），服务只读、仅监听本机。

2. 三个真实的协同问题与纠偏

① Q 矩阵缺 KP5 测量覆盖。 初版没有「仅测 KP5」的题，导致「全未掌握」与「仅掌握 KP5」两种画像产生相同 理想作答，32 种画像无法两两区分。独立复核发现后，锁定 24 题 = 三份 I5（含仅测 KP5 的三道）+ 9 道综合，并补 测试验证 32 画像理想反应向量两两互异；随后又把 24 条题干改成互不相同，KP5 单技能只考列式并给出算术结果、 KP3 单技能若需通分则直接给出等价结果，避免把实际需要的分数运算悄悄标成单技能。

② 弱测试允许「伪实现」。 最初的 fit 测试只断言返回结构、边界与收敛标志，可被「始终返回初始 g/s、history 平坦、converged=False」的伪实现通过--它根本没跑 EM。独立复核发现后，新增一步 EM 手算 fixture（K=1、Q=[[1]]、X=[[1],[1],[1],[0]]，断言 max_iter=1 实际更新题参且似然严格提高，其中 g=27/71 截断到 0.35、s=11/254），并断言最终后验边际与用最终题参重算逐项一致，把「真的在优化」做成可检验的契约。这两个分数是 独立手算的期望计数比，不调用待测代码，充当独立判据。

③ Claude 的 Git/HTTP 沙箱与外部验证。 实测确认 Claude 沙箱内 .git 写入、本地端口绑定 （socket.bind → EPERM）、浏览器进程注册都受限。据此明确分工：文档/代码/测试由 Claude 生成，git 提交与 沙箱外运行测试/浏览器由外部编排（Codex）完成；沙箱内无法跑通的 HTTP/浏览器项如实标注，绝不伪装成通过。 外部首轮浏览器验收 12/13（仅 B7 的 End 键在 macOS 原生 select 不生效），定位为平台差异后为 select 显式处理 Home/End，复跑后 15 组全绿；最终 79 项 Python、15 组前端、EM 33 轮收敛全部落盘留证。

3. 对算法工程与 AI 提效的理解

算法工程上，数值稳定（对数域 logsumexp）、可测试契约（手算 fixture）、数据链一致性（从库读回 Q/X 重跑真 EM 逐题比对）比「跑得通」更重要--它们把正确性钉死在可复现的断言上，而不是靠一次演示截图。AI 提效上，测试先行 加契约冻结能把「意图」固化下来：把 data-testid、数值格式、几何契约写进测试，就成了一份可执行的活规格，让模型 生成可核验的实现并快速走完 RED→GREEN。设计阶段冻结的 API 契约、EM 数值契约与前端 data-testid 契约是贯穿各阶段 的唯一事实来源，避免设计与实现漂移。但模型仍会留下盲区（伪实现、测量覆盖缺口），需要独立复核 + 人工纠偏。 AI 生成 ≠ 已验证：外部干净环境的冒烟验收和真实截图核验不可省，测试次数与真实证据也必须留痕。若重做， 会把 UI 回归也尽早纳入初始契约、减少 QA 阶段的追加，并在更早引入独立复核，避免伪实现与测量覆盖这类盲区拖延到后期。

4. 交付与验证

源码（src/ + static/）、测试（tests/）、文档（docs/）均已就绪。外部验收：79 项 Python unittest 全过、 15 组前端验收全过（6 HTTP + 9 浏览器）、演示 EM 33 轮收敛，桌面与 390px 截图由 Codex 查看通过。各阶段 RED/GREEN 原始命令与退出码存 docs/validation/，真实 git 历史体现 docs/schema → tests(RED) → feat(GREEN) → E2E。

5. 真实分工（不虚构）

用户提出并授权任务、确认需求取舍；Codex 及其审查代理负责技术拆解、独立复核、纠偏提示与操作编排（代操作、 本地 git 编排、沙箱外 QA 与截图查看）；代码、测试、文档由 Claude Code 会话中配置的 **DeepSeek** 模型生成， 并真实调用 Superpowers 工作流。本文不声称用户手写或手测过代码/测试；模型路由依据已验证配置与 DeepSeek 官方 兼容别名映射，模型自称本身不作为独立身份证明。